

USIL

Universal SHA-256 Interoperability Layer

Kaspa L1 Native Protocol Primitive | Whitepaper v3.0

Kaspa Toccata Hardfork: June 5–20, 2026

USIL is a native Kaspa L1 protocol primitive — not a contract on top of Kaspa. A sovereign vProg that executes off-chain and settles SHA-256 state commitments directly into Kaspa consensus via Groth16 ZK verification, minting synthetic cross-chain assets at Kaspa consensus-level security. sBTC that is as secure as KAS.

Status	Version	Architecture	Hardfork Required
Public Draft	v3.0 — 2026	Kaspa L1 vProg + CDAG	Toccata (Jun 2026)

Table of Contents

1. Executive Summary.....	3
2. Problem Statement.....	3
3. The Invention: What USIL Actually Is.....	4
4. Why L1 Native — Not a Contract.....	5
5. Commitment Specification.....	6
6. CDAG Integration.....	7
7. Proof System Adapters + ZK Pipeline.....	8
8. Trust Models at L1.....	9
9. Why Kaspas / The SHA-256 Alignment.....	10
10. Tokenization Model — Native KRC-20.....	11
11. Threat Model.....	12
12. Concrete Flow: BTC → Kaspas L1.....	13
13. Development Roadmap.....	14
14. Tech Stack.....	15
15. Open Challenges.....	15
16. Conclusion.....	16

1. Executive Summary

USIL v3.0 is a significant architectural evolution from v2.0. Where v2 positioned USIL as a Kasplex smart contract layer, v3 implements USIL as a native Kaspas L1 protocol primitive — a sovereign vProg that settles directly into Kaspas consensus state, enabled by the Toccata hardfork activating June 5–20, 2026.

The core shift: synthetic sBTC minted by USIL v3 inherits Kaspas consensus-level security — the same guarantee that native KAS holds. Not Kasplex VM security. Not contract security. Kaspas consensus security.

This is made possible by three Toccata primitives: vProgs (sovereign off-chain programs with on-chain settlement), native Groth16 ZK verification at the L1 opcode level, and true native KRC-20 tokens on L1. USIL uses all three in a single coherent architecture.

The reference implementation — Python simulation engine, Ghost→Shadow→Live pipeline, SQLite MintLedger, and 6-attack threat model validator — is already running. The vProg spec is submitted. When Toccata activates, USIL deploys.

2. Problem Statement

2.1 Ecosystem Fragmentation

Blockchain ecosystems remain deeply fragmented across incompatible consensus mechanisms, hashing algorithms, and state models. Bitcoin (UTXO + SHA-256 + PoW), Ethereum (account + Keccak + PoS), and Kaspas (DAG + SHA-256 + PoW) cannot natively read each other's state. Billions in assets are siloed with no trustless path between them.

2.2 The Deeper Problem: Security Inheritance

Existing bridges do not just fail at verification — they fail at security inheritance. A wrapped BTC on a smart contract chain is only as secure as that chain's VM and the contract code itself. A smart contract bug, a VM exploit, or a governance attack can compromise billions in bridged assets regardless of how sound the underlying Bitcoin security is.

The correct question is not 'how do we move assets between chains?' It is 'how do we move assets between chains such that the synthetic representation inherits the security of the destination chain's consensus — not just its VM?'

2.3 Bridge Failure Modes

Bridge Type	Security Bound	Real-World Failure
Custodial	Operator honesty	FTX, Ronin (\$625M)
MPC Multisig	Signer collusion resistance	Multichain collapse (2023)
Optimistic (contract)	Contract correctness	Smart contract exploits
Light Client (contract)	VM + contract correctness	Contract upgrade attacks
USIL v3 (L1 native)	Kaspa consensus	L1 consensus failure only

3. The Invention: What USIL Actually Is

USIL is a standardized cross-chain state serialization protocol with pluggable ZK proof backends, implemented as a Kaspa native vProg. SHA-256 is the commitment envelope. Groth16 is the L1 verification mechanism. CDAG is the permanent state record. Native KRC-20 is the settlement token — secured by Kaspa consensus.

This framing has three important implications. First, SHA-256 does not provide security alone — it provides a deterministic, chain-agnostic commitment format. Security comes from the ZK proof verified by the Groth16 L1 opcode. Second, the commitment format and the proof system are strictly separated — upgrading from SPV to native Groth16 proving does not change the commitment format. Third, settlement happens at L1 — not in a VM, not in a contract.

3.1 Full System Architecture

Layer	Location	Components	Security Bound
L0 — Source Chains	External	Bitcoin, Ethereum, any chain	Own consensus
L1 — Extraction	Off-chain	State readers, SPV provers	Network + RPC
L2 — Commitment	Off-chain	SHA-256 engine, ZK prover	SHA-256 + Groth16
L3 — L1 Settlement	Kaspa L1	vProg, CDAG, Groth16 opcode	Kaspa consensus
L4 — Native Mint	Kaspa L1	SilverScript covenant, KRC-20	Kaspa consensus

3.2 What Changes from v2 to v3

Component	v2 (Kasplex Contract)	v3 (L1 Native vProg)
Commitment registry	Solidity contract	CDAG L1 state
Proof verification	Contract function	Groth16 L1 opcode
MintLedger	Contract storage	CDAG state (consensus-enforced)
KRC-20 token	Kasplex overlay	True native L1 token
sBTC security	Kasplex VM level	Kaspa consensus level
Upgrade path	Contract upgrade	L1 governance + timelock
Audit trail	Contract events	CDAG — permanent, blue-score anchored

4. Why L1 Native — Not a Contract

This is the architectural decision that separates USIL v3 from every other bridge protocol in the Kaspas ecosystem. The argument has three layers.

4.1 Security Inheritance

A Kaspex contract has its security bounded by the Kaspex VM. Any vulnerability in the VM, any bug in the contract, any governance decision about Kaspex affects the security of every synthetic asset the contract has minted. The attack surface includes the VM, the contract bytecode, the contract upgrade mechanism, and the governance process that controls upgrades.

A vProg settling into CDAG has its security bounded by Kaspas consensus. The attack surface is reduced to: (1) the ZK proof system soundness, (2) the SilverScript covenant correctness, and (3) Kaspas consensus itself. There is no VM layer, no contract upgrade mechanism, no intermediate governance layer.

4.2 Permanence

Contract storage can be modified by contract upgrades. CDAG state is permanent — every commitment, every mint record, every BurnReceipt is anchored to a Kaspas blue score and cannot be altered by any party, including the USIL team. This is the correct property for a financial settlement system.

4.3 The vProg Model is Exactly Right

vProgs are sovereign programs that execute outside L1 but settle results back on-chain through proofs. — Kaspas core developer description of vProgs. USIL's execution model: extract state off-chain, generate ZK proof off-chain, settle commitment on-chain. This is not USIL adapting to fit the vProg model. This is the vProg model describing USIL.

5. Commitment Specification

5.1 Base Commitment Format (unchanged)

```
commitment := VERSION (1 byte = 0x01) ■ SHA256( chain_id (4 bytes, big-endian uint32) ■  
block_height (8 bytes, big-endian uint64) ■ state_root (32 bytes) )
```

5.2 ZK Extension for L1 Settlement

L1 settlement requires a Groth16 proof accompanying every commitment. The proof encodes correctness of the commitment preimage — proving that the state_root was correctly extracted from the source chain without revealing the full extraction witness on-chain.

```
zk_payload := { commitment: bytes33, // VERSION ■ SHA256(preimage) groth16_proof:
bytes200, // constant-size ZK proof public_inputs: [ chain_id, // registered chain
block_height, // within validity window commitment_hash, // matches commitment field ],
blue_score_ref: uint64, // Kaspa finality anchor }
```

5.3 CDAG Execution Commitment

```
execution_commitment := { program_id: USIL_VPROG_ID, dependencies:
[prev_commitment_hash], // chain continuity resource_usage: { compute: N, storage: M },
blue_score: current_kaspa_blue_score, }
```

The execution commitment embeds USIL's commitment history into Kaspa's CDAG as a sub-DAG. Every commitment is traceable to a specific Kaspa blue score — providing a global, trustless, consensus-anchored timestamp for every cross-chain state observation USIL has ever made.

6. CDAG Integration

The Computational DAG (CDAG) is Kaspa's system for tracking program resource usage, execution dependencies, and program commitments across all vProgs. USIL commitments form a sub-DAG within the global CDAG.

6.1 USIL Sub-DAG Structure

```
CDAG (Kaspa global) ■■■ USIL commitment sub-DAG ■■■ BTC block 892,001 @ blue_score N ■
■■■ ZK proof: groth16_verify() = true ■ ■■■ mint: 0.1 sBTC → kaspa:qr9y... ■■■ ETH
block 21,450,000 @ blue_score N+12 ■ ■■■ ZK proof: groth16_verify() = true ■ ■■■
mint: 0.5 sETH → kaspa:qr9y... ■■■ BTC block 892,002 @ blue_score N+18 ■■■ depends
on: BTC block 892,001 (chain continuity) ■■■ ZK proof: groth16_verify() = true
```

6.2 Blue Score Finality

USIL uses Kaspa's blue score as its universal finality reference. A commitment is considered final when the current blue score exceeds the commitment's blue score reference by the `FINALITY_THRESHOLD`. At 10 BPS, this means:

Source Chain	Source Finality	USIL Finality (Kaspa)	Speed Gain
Bitcoin	~60 min (6 blocks)	~10 seconds	360x faster
Ethereum	~12.8 min (2 epochs)	~10 seconds	76x faster
Kaspa	~10 seconds (native)	~10 seconds	Native

The source chain's finality still governs when a commitment can be submitted — Bitcoin commitments still require 6 confirmations (~60 min). But once submitted to Kaspa L1, the USIL commitment itself reaches Kaspa-level finality in ~10 seconds.

7. Proof System Adapters + ZK Pipeline

Every proof adapter implements a common interface that outputs a Groth16 proof consumable by the Kaspas L1 Groth16 opcode. The inner verification logic varies per source chain; the output format is always identical.

```
interface ProofAdapter { generate(block_header, state_root) → Groth16Proof
  verify(commitment, proof) → bool circuit_id() → bytes32 finality_blocks() → uint64 }
```

7.1 Bitcoin — SPV → Groth16

The Bitcoin SPV adapter wraps classical SPV verification inside a Groth16 circuit. The circuit proves: (a) the block header satisfies Bitcoin PoW difficulty, (b) the lock transaction is included in the block's Merkle tree. The L1 Groth16 opcode verifies the wrapped proof — no raw SPV on-chain.

7.2 Ethereum — Merkle Patricia → Groth16

The Ethereum adapter wraps Patricia trie traversal in a Groth16 circuit. The circuit proves the state_root correctly encodes the claimed account state. Requires 2 epochs (economic finality) before commitment submission.

7.3 Native Groth16 (Phase 3)

Phase 3 eliminates the SPV/Patricia wrapper entirely. A SNARK circuit directly encodes the source chain's state transition function. The prover generates a single Groth16 proof covering all blocks since the last commitment. Constant proof size. Constant L1 verification cost. No trust assumptions beyond ZK proof soundness.

8. Trust Models at L1

All three trust modes settle into Kaspas L1 CDAG state. The difference is what evidence is required before settlement is accepted.

Mode	Evidence Required	L1 Enforcement	Speed	Best For
TRUSTLESS	Groth16 ZK proof	L1 opcode — consensus	~10s post-finality	High-value
OPTIMISTIC	Commitment + challenge window	SilverScript covenant	7-day window	Mid-value
ORACLE	2/3 validator signatures	SilverScript quorum check	<30 seconds	Low-value, fast

8.1 Why Even Oracle Mode is Stronger at L1

In a Kasplex contract, an oracle validator set could potentially coordinate a contract upgrade to cover fraud tracks. At Kaspas L1, every oracle submission is permanently recorded in CDAG state — anchored to a blue score, immutable, publicly auditable by any Kaspas node. Oracle fraud is permanently visible in L1 state even if the on-chain slashing mechanism is somehow bypassed.

9. Why Kaspas — The SHA-256 Alignment

Three systems. One hash function. All at L1. Bitcoin PoW → SHA-256 (consensus-level) Kaspas PoW → SHA-256 (consensus-level) USIL → SHA-256 (commitment-level) This is not a marketing claim. It is a cryptographic property that makes USIL natively verifiable by participants in both networks using hardware and tooling they already operate.

9.1 What the Alignment Means

- **Bitcoin block headers can be committed without format translation** — the same SHA-256 hash that Bitcoin miners compute is the same function USIL uses for commitment generation
- **Bitcoin mining hardware validates Kaspas** — economic alignment between the two largest SHA-256 networks, creating natural incentive overlap for USIL validators
- **USIL commitments are natively auditable** — any Bitcoin or Kaspas full node can independently verify a USIL commitment's preimage correctness without additional tooling
- **Post-quantum path is shared** — if SHA-256 requires replacement, Bitcoin, Kaspas, and USIL upgrade together under the same research umbrella

9.2 Toccata Hardfork — What It Delivers for USIL

USIL Requirement	Toccata Primitive	Impact
Off-chain exec + L1 settlement	vProgs	Core architecture unlocked
ZK verification at consensus	Groth16 L1 opcode	Trustless mode enabled
L1 security for synthetics	Native KRC-20 on L1	sBTC = KAS security level
Execution dependency tracking	CDAG	Permanent audit trail
Programmable mint conditions	Covenants++ / SilverScript	MintLedger at L1
Appchain / rollup settlement	vProgs	Future multi-chain expansion

9.3 DAGKnight Alignment (Post-Toccata)

DAGKnight — the successor to GHOSTDAG — introduces a no-delay-bound consensus model with faster convergence in transaction ordering. USIL's finality model is designed to upgrade to DAGKnight ordering once available, achieving even faster commitment finality and deterministic ordering for high-throughput bridge scenarios.

10. Tokenization Model — Native KRC-20

USIL v3 mints synthetic assets as true native KRC-20 tokens on Kaspa L1 — not Kasplex overlay tokens. This distinction has concrete security implications.

10.1 Mint Conditions (SilverScript Covenant)

```
covenant USILMint { require: commitment in CDAG_state require:
groth16_verify(commitment.proof) == true require: blue_score_ref + FINALITY_THRESHOLD
<= current_blue_score require: block_height within 2016_block_validity_window require:
NOT mint_ledger[commitment_id][utxo_id].exists effect:
mint_ledger.insert(commitment_id, utxo_id, amount, dest) krc20_native.mint(sBTC,
amount, dest) }
```

10.2 Security Properties of Native sBTC

Property	v2 (Kasplex Contract)	v3 (L1 Native)
Mint validity	Contract correctness	Kaspa consensus + Groth16
Double-mint prevention	Contract storage mapping	CDAG state (consensus-enforced)
Expiry enforcement	Contract function check	SilverScript covenant
Transfer security	Kasplex VM	Kaspa L1 consensus
Upgrade risk	Contract upgrade path	L1 governance + timelock

10.3 Redemption Flow

To redeem: holder burns sBTC on Kaspa L1 → L1 emits BurnReceipt commitment into CDAG → vProg generates ZK proof of BurnReceipt → any trustless relayer submits proof to source chain lock contract → original asset released. The relayer is fully permissionless — no whitelist, no registration, no trust.

11. Threat Model

[T1] Invalid State Root Severity: Critical

Attack: Fabricated state_root submitted to CDAG.

Mitigation: Groth16 L1 opcode rejects — ZK proof cannot be generated for a false preimage. Attack is computationally infeasible (ZK soundness assumption).

[T2] Oracle Collusion Severity: High

Attack: 2/3+ oracle validators collude to submit false commitment.

Mitigation: SilverScript quorum check enforced at L1. Fraud permanently visible in CDAG state even if quorum is met. Validator stake slashed by SilverScript covenant.

[T3] Double Mint Severity: Critical

Attack: Attacker mints sBTC twice for same locked UTXO.

Mitigation: CDAG MintLedger is append-only, consensus-enforced. Second mint attempt fails at L1 — no contract to exploit, no storage to manipulate.

[T4] Replay / Reorg Severity: Medium

Attack: Source chain reorgs — orphaned block commitment reused.

Mitigation: 6-confirmation SPV requirement. Blue score finality threshold. Orphaned commitments auto-invalidated by CDAG dependency tracking.

[T5] Stale Commitment Severity: High

Attack: Old valid commitment used to mint after UTXO spent.

Mitigation: SilverScript covenant enforces 2,016-block expiry window at L1. Expired commitments rejected unconditionally by covenant logic.

[T6] Proof System Bug Severity: Critical

Attack: Bug in ZK circuit allows invalid proof to pass Groth16 opcode.

Mitigation: Circuit audits required before deployment. 48-hour L1 governance timelock for circuit upgrades. Bug bounty active from Phase 2 launch.

12. Concrete Flow: BTC → Kaspal L1

Step 1 Lock on Bitcoin

User sends 0.1 BTC to the USIL lock address. OP_RETURN output encodes the destination Kaspal address. Lock address is controlled by a SilverScript covenant — released only on valid BurnReceipt proof.

Step 2 SPV Prover Monitors

USIL SPV prover watches Bitcoin. After 6 confirmations, extracts: chain_id=0x00000001, block_height=N, merkle_root of block N.

Step 3 Commitment Generated

commitment = SHA256(0x00000001 ■ N ■ merkle_root). Groth16 prover generates ZK proof over the commitment preimage — proving SPV correctness without revealing full Merkle witness on-chain.

Step 4 vProg Submits to CDAG

USIL vProg submits zk_payload to Kaspal L1: commitment (33 bytes) + Groth16 proof (~200 bytes) + execution_commitment (CDAG accounting). Gas cost: minimal KAS — Kaspal's 10 BPS makes this economically trivial.

Step 5 Groth16 Opcode Verifies

Kaspal L1 executes: groth16_verify(proof, public_inputs). If valid: commitment written to CDAG state at current blue score. If invalid: rejected at consensus level — no state change, no fee loss.

Step 6 Blue Score Finality

Commitment reaches Kaspal finality in ~10 seconds (FINALITY_THRESHOLD blue score units at 10 BPS). This is independent of Bitcoin's 60-minute finality — the commitment is Kaspal-final once the blue score clears.

Step 7 SilverScript Mint

USILMint covenant executes: all conditions checked at L1. MintLedger updated in CDAG state (append-only, consensus-enforced). 0.1 native sBTC (KRC-20 L1) minted to the user's Kaspal address.

Step 8 User Receives Native sBTC

User's Kaspal wallet shows 0.1 sBTC as a native L1 token — same security as KAS. Transferable, stakeable, DeFi-composable. Total time: ~70 minutes (dominated by 6 BTC confirmations). Kaspal settlement: ~10 seconds after vProg submission.

Reverse (sBTC → BTC): Holder burns native sBTC on Kaspas L1 → L1 emits BurnReceipt into CDAG → vProg generates Groth16 proof → Any relayer submits proof to Bitcoin lock covenant → 0.1 BTC released. Total time: ~15 minutes. Fully trustless.

13. Development Roadmap

Phase	Name	Deliverables	Status
1 — Now	Spec + Sim	vProg spec published. Python simulation running. Ghost→Shadow→Live + CDAG layer. All 6 attacks caught.	Complete
2 — Toccata	L1 Deploy	USIL vProg deployed on Kaspas mainnet (Jun 2026). BTC→Kaspas bridge on testnet. Native sBTC mint. SPV→Groth16 prover.	5 Weeks
3 — Bridge Alpha	BTC Live	Mainnet BTC→Kaspas bridge. Security audit. Bug bounty launch. SilverScript covenant deployment. CDAG MintLedger live.	Q3 2026
4 — ETH Bridge	Multi-Asset	Ethereum Merkle Patricia adapter. sETH native token. Optimistic mode. Validator staking contracts on L1.	Q4 2026
5 — ZK Native	Trustless	Direct Groth16 proving — no SPV wrapper. Sub-5-minute BTC→Kaspas with full ZK trustlessness. DAGKnight finality alignment.	Q2 2027
6 — Ecosystem	Standard	Multi-chain registry governance. USIL SDK (Python, Rust, JS). Developer grants. KIP proposal for USIL as Kaspas canonical bridge.	Q3 2027

14. Tech Stack

Component	Technology	Purpose
Commitment Engine	Python 3.11+ / Rust	SHA-256 generation, state extraction
ZK Prover	Circom + SnarkJS → Groth16	ZK proof generation for L1 opcode
SPV Prover	Python + bitcoinlib	Bitcoin header tracking, Merkle proofs

vProg Runtime	Kaspa vProg SDK (post-Toccata)	Off-chain execution, L1 submission
Mint Covenant	SilverScript	USILMint conditions, MintLedger
Native Token	KRC-20 L1 (Toccata native)	sBTC, sETH — Kaspa consensus security
Simulation	Python + Rich + SQLite	Ghost→Shadow→Live demo, pre-Toccata
Dashboard	HTML/JS + Python server	Visual pipeline monitor
Infrastructure	Docker + systemd	Raspberry Pi 5 or cloud deployable

15. Open Challenges

■ ZK Proving Cost

Groth16 proof generation for Bitcoin SPV is computationally intensive (~30-120 seconds on standard hardware). Recursive proof composition (Nova, folding schemes) may reduce this significantly. Community research welcome.

■ Bitcoin Lock Covenant

Bitcoin script cannot verify arbitrary proofs on-chain. The Phase 2 lock address uses a multisig — partially custodial. Full trustless Bitcoin lock requires BitVM or OP_CTV covenant mechanisms not yet live on Bitcoin mainnet.

■ vProg SDK Maturity

The Kaspa vProg SDK is new post-Toccata. USIL requires close coordination with Kaspa core developers during Phase 2 integration. Early access to Toccata testnet is being sought.

■ CDAG Storage Pricing

Optimal pricing for USIL commitment storage in the CDAG is an open question. Per-byte vs. flat fee vs. proportional to source chain complexity. Seeking input from Kaspa economic researchers.

■ MEV Resistance

As USIL grows, large bridge transactions may attract MEV in Kaspa's emerging MEV landscape. Commitment submissions should be designed to be front-running resistant — design TBD pending Kaspa MEV specification.

16. Conclusion

USIL v3 is the right protocol at the right moment. The Toccata hardfork delivers exactly the L1 primitives required — vProgs, Groth16 opcode, native KRC-20 — and USIL is designed to use all three in a coherent

architecture that settles cross-chain state directly into Kaspas consensus.

The result is the strongest security model available for a cross-chain bridge in the Kaspas ecosystem: synthetic sBTC that inherits Kaspas consensus security, a MintLedger enforced by consensus rather than contract logic, and an audit trail permanently embedded in the CDAG.

USIL is not built on Kaspas. It is built into Kaspas. The SHA-256 alignment between Bitcoin, Kaspas, and USIL is not marketing — it is the cryptographic foundation of an interoperability protocol that operates at the same layer as both chains it bridges. Toccata activates in 5 weeks. The implementation is ready.

github.com/usil-protocol | usil.io | MIT License | v3.0 — 2026 | Kaspas L1 Native Protocol